GOVERNO DO ESTADO DO CEARÁ • SEDUC

Educação Profissional • Eixo Informação e Comunicação

Operadores e Introdução a Funções

PROGRAMAÇÃO EM PYTHON

Atribuição, comparação e os primeiros passos com funções

E.E.E.P. Lúcia Helena Viana Ribeiro — Horizonte/CE
Prof. Francisco Ericson Cornélio da Costa

AULA DE HOJE

O que vamos ver

1. **Operadores de atribuição**
   = e as formas compostas
2. **Operadores de comparação**
   Resultado sempre True ou False
3. **= ou == ?**
   A diferença que mais causa erro
4. **Exemplos resolvidos**
   Passo a passo, com teste de mesa
5. **Introdução a funções**
   Dar nome a um pedaço de programa

1

PARTE

Operadores de atribuição

Guardar e atualizar valores

ATRIBUIÇÃO

O operador = guarda um valor

O sinal = NÃO significa igualdade em programação. Ele significa: guarde o valor da direita na variável da esquerda.

programa.py

```
idade = 15

print(idade)
```

TERMINAL

```
15
```



o valor vai para a variável

COMO LER

Leia idade = 15 como "idade RECEBE 15", nunca como "idade é igual a 15".

ATRIBUIÇÃO

O valor pode mudar

programa.py

```
nota = 7
print(nota)

nota = 9
print(nota)
```

TERMINAL

```
7
9
```

- A segunda atribuição SUBSTITUI a primeira: o valor antigo se perde.
- Por isso a variável se chama variável — o conteúdo varia.

GUARDE ISTO

Vale sempre o último valor atribuído.

ATRIBUIÇÃO

Usando a própria variável

Uma variável pode aparecer dos dois lados do =. O Python calcula a direita primeiro e só depois guarda.

programa.py

```
pontos = 10
pontos = pontos + 5
print(pontos)
```

TERMINAL

```
15
```

Como o Python resolve

```
1) calcula a direita:  pontos + 5  ->  10 + 5  ->  15
2) guarda na variável: pontos  <-  15
```

As formas compostas

Atualizar uma variável com ela mesma é tão comum que existe um atalho para cada operação.

Atalho	Equivale a	Exemplo (x = 10)	Fica
`x += 3`	`x = x + 3`	`10 + 3`	`13`
`x -= 3`	`x = x - 3`	`10 - 3`	`7`
`x *= 3`	`x = x * 3`	`10 * 3`	`30`
`x /= 4`	`x = x / 4`	`10 / 4`	`2.5`
`x //= 4`	`x = x // 4`	`10 // 4`	`2`
`x %= 4`	`x = x % 4`	`10 % 4`	`2`
`x **= 2`	`x = x ** 2`	`10 ** 2`	`100`

IMPORTANTE

O atalho faz exatamente a mesma coisa: calcula e regrava na mesma variável.

Exemplo resolvido: teste de mesa

programa.py

```
saldo = 100
saldo -= 30
saldo += 10
saldo *= 2
print(saldo)
```

Linha	Operação	saldo
1	saldo = 100	100
2	saldo - 30	70
3	saldo + 10	80
4	saldo * 2	160

TERMINAL

```
160
```

TESTE DE MESA

Acompanhe o valor da variável linha por linha: é assim que se confere um programa sem rodar.

2

PARTE

Operadores de comparação

A resposta é sempre True ou False

COMPARAÇÃO

Comparar é fazer uma pergunta

Um operador de comparação pergunta algo ao computador. A resposta nunca é um número: é True (verdadeiro) ou False (falso).

programa.py

```
idade = 15

print(idade > 18)
print(idade < 18)
```

TERMINAL

```
False
True
```

GUARDE ISTO

Toda comparação produz um valor do tipo bool: True ou False.

COMPARAÇÃO

Os seis operadores

Operador	O que pergunta	Exemplo	Resultado
`>`	é maior que?	`8 > 3`	`True`
`<`	é menor que?	`8 < 3`	`False`
`>=`	é maior ou igual a?	`8 >= 8`	`True`
`<=`	é menor ou igual a?	`3 <= 8`	`True`
`==`	é igual a?	`8 == 8`	`True`
`!=`	é diferente de?	`8 != 3`	`True`

ATENÇÃO AO LIMITE

Os operadores >= e <= incluem o limite: 8 >= 8 é verdadeiro.

COMPARAÇÃO

Comparando textos

programa.py

```
senha = "abc123"

print(senha == "abc123")
print(senha == "ABC123")
```

TERMINAL

```
True
False
```

POR ISSO SENHA DIFERENCIA

Em texto, maiúscula e minúscula são caracteres diferentes.

É por esse motivo que digitar a senha com Caps Lock ligado não funciona.

3

PARTE

= ou == ?

A diferença que mais causa erro

ATENÇÃO

Um sinal muda tudo

=

ATRIBUIÇÃO

Guarda um valor na variável.
Não responde nada.

```
idade = 18
```

==

COMPARAÇÃO

Pergunta se os dois são iguais.
Responde True ou False.

```
idade == 18
```

EM UMA FRASE

Um sinal = guarda. Dois sinais == perguntam.

ATENÇÃO

Vendo a diferença rodando

programa.py

```
idade = 18
print(idade)

print(idade == 18)
print(idade == 20)
```

TERMINAL

```
18
True
False
```

- A linha idade = 18 guarda o número e não mostra nada sozinha.
- As linhas com == comparam e devolvem True ou False.

CUIDADO NO IF

Trocar == por = dentro de um if faz o programa dar erro.

4

PARTE

Exemplos resolvidos

Juntando tudo

EXEMPLO RESOLVIDO

Controle de pontos de um jogo

programa.py

```
pontos = 0

pontos += 50      # acertou uma fase
pontos += 30      # acertou outra
pontos -= 20      # perdeu uma vida

print(pontos)
print(pontos >= 100)
```

TERMINAL

```
60
False
```

- 3 atualizações
- 1 comparação

REPARE NA ORDEM

Primeiro o programa atualiza a variável; só no fim ele compara e responde.

EXEMPLO RESOLVIDO

Média e aprovação

programa.py

```
nota1 = 8
nota2 = 6

media = (nota1 + nota2) / 2

print(media)
print(media >= 7)
```

TERMINAL

```
7.0
True
```

- = calcula e guarda
- == ou >= pergunta

ATENÇÃO AO LIMITE

A média 7.0 satisfaz >= 7, porque o sinal >= inclui o próprio 7.

PRATIQUE

Qual é a saída?

Questão 1

```
x = 20
x //= 3
print(x)
```

Questão 2

```
y = 7
print(y == 7)
print(y != 7)
```

Questão 3

```
z = 5
z **= 2
print(z > 20)
```

Questão 4

```
a = 10
a %= 4
print(a)
```

Responda no caderno antes de digitar no computador.

PARTE

5

Introdução a funções

Dar um nome a um pedaço de programa

FUNÇÕES

Você já usa funções desde a primeira aula

Toda vez que você escreve um nome seguido de parênteses, está chamando uma função. Elas já estavam lá.

Função	O que faz	Exemplo
`print()`	mostra algo na tela	`print("Olá")`
`input()`	lê o que o usuário digita	`input("Idade: ")`
`int()`	converte para número inteiro	`int("15")`
`len()`	conta os caracteres de um texto	`len("Ceara")`
`type()`	informa o tipo de um valor	`type(idade)`

DEFINIÇÃO

Função é um pedaço de programa que já tem nome e faz um trabalho para você.

FUNÇÕES

As partes de uma chamada

Exemplo

```
tamanho = len("Ceara")
```

NOME

`len`

qual função será usada

ARGUMENTO

`"Ceara"`

o que entra, entre parênteses

RETORNO

`5`

o valor que ela devolve

O CICLO

A função recebe algo, faz o trabalho e devolve um resultado.

Por que criar as suas próprias

Repetindo o cálculo

```
n1 = 8
n2 = 6
m1 = (n1 + n2) / 2

n3 = 7
n4 = 9
m2 = (n3 + n4) / 2

# e assim por diante...
```

Com uma função

```
def media(a, b):
    return (a + b) / 2

m1 = media(8, 6)
m2 = media(7, 9)
m3 = media(10, 4)
```

- A conta é escrita UMA vez e usada quantas vezes precisar.
- Se a regra mudar, você corrige em um lugar só.

A IDEIA CENTRAL

Função serve para não repetir o mesmo trabalho.

Criando a sua primeira função

programa.py

```
def saudacao():
    print("Bom dia, turma!")

saudacao()
```

TERMINAL

```
Bom dia, turma!
```

`def`	palavra que inicia a definição
`saudacao`	o nome que você escolhe
`( )`	onde entram os dados, se houver
`:` e recuo	delimitam o corpo da função

ATENÇÃO

Definir a função não executa nada. Ela só roda quando é chamada pelo nome.

Passando informação: parâmetros

programa.py

```
def saudacao(nome):
    print("Bom dia,", nome)

saudacao("Maria")
saudacao("João")
```

TERMINAL

```
Bom dia, Maria
Bom dia, João
```

- O nome entre parênteses na definição é o PARÂMETRO: um espaço reservado.
- O valor enviado na chamada é o ARGUMENTO: preenche esse espaço.

REPARE

A mesma função serve para qualquer nome: muda só o que você envia.

Devolvendo um resultado: return

programa.py

```
def dobro(numero):
    return numero * 2

resultado = dobro(7)
print(resultado)
print(dobro(10))
```

TERMINAL

```
14
20
```

- O return entrega um valor de volta para quem chamou a função.
- Esse valor pode ser guardado em uma variável ou usado direto.

IMPORTANTE

Depois do return a função termina: nada abaixo dele é executado.

print ou return?

print

MOSTRA NA TELA

```
def dobro(n):
    print(n * 2)
```

O valor aparece, mas não pode ser reaproveitado no programa.

return

DEVOLVE O VALOR

```
def dobro(n):
    return n * 2
```

O valor volta e pode ser guardado, somado, comparado, reutilizado.

A DIFERENÇA

print só exibe. return entrega o valor de volta para o programa usar.

Exemplo resolvido: situação do aluno

programa.py

```
def media(n1, n2):
    return (n1 + n2) / 2

m = media(8, 6)

print(m)
print(m >= 7)
```

TERMINAL

```
7.0
True
```

- return devolve 7.0
- o >= compara depois

TUDO SE JUNTA

A função faz o cálculo; os operadores da aula de hoje continuam valendo lá dentro.

PRATIQUE

Funções

Questão 5 — qual a saída?

```
def triplo(n):
    return n * 3

print(triplo(5))
```

Questão 6 — qual a saída?

```
def soma(a, b):
    return a + b

x = soma(4, 6)
print(x > 8)
```

Questão 7 — escreva a função

```
# Escreva uma função chamada quadrado que recebe um
# número e devolve esse número elevado ao quadrado.
```

Responda no caderno antes de digitar no computador.

RESUMO

O que levar desta aula

=

Guarda um valor na variável. Lê-se RECEBE.

+= -= *= /=

Atalhos que atualizam a própria variável.

//= %= **=

Divisão inteira, resto e potência, também como atalho.

> < >= <=

Comparam tamanho. Os com = incluem o limite.

== e !=

Perguntam se é igual ou diferente. Devolvem True ou False.

def

Cria uma função: dá nome a um pedaço de programa.

return

Devolve o resultado da função para o programa usar.

Um sinal guarda.
Dois sinais perguntam.

```
=  guarda      ==  pergunta      def  dá nome ao que se repete
```